ANEXO LABORATORIO 3

David Santiago Morales Romero, Miguel Angel Benavides Walteros, Daniel Mauricio Beltran Romero

1. Arquitectura del sistema

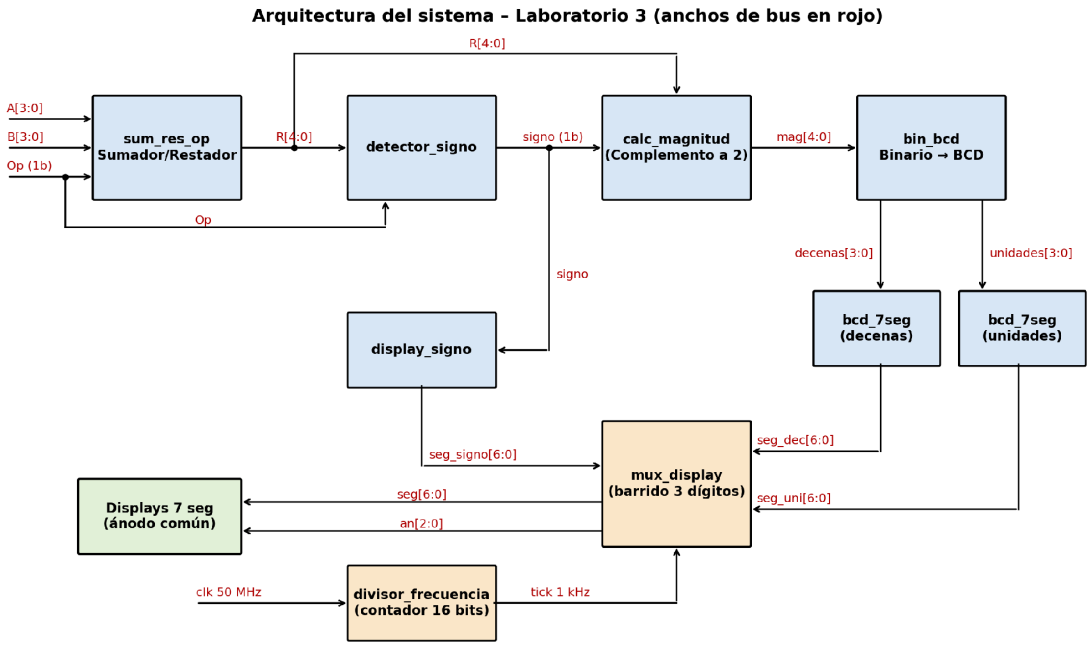

Figura 1. Diagrama de bloques del sistema con anchos de bus rotulados. A[3:0], B[3:0] y Op entran al sumador/restador, que entrega R[4:0]. detector_signo evalúa la polaridad y calc_magnitud extrae mag[4:0] en valor absoluto. bin_bcd entrega decenas y unidades hacia los decodificadores bcd_7seg, y display_signo decodifica el signo. El multiplexor envía la información al bus común seg[6:0] sincronizado por el divisor de frecuencia (tick de 1 kHz) y habilita secuencialmente cada display con an[2:0].

2. Casos de prueba y valores esperados en buses internos

Operación	Op	R[4:0]	signo	mag[4:0]	dec	uni	Display Signo
3 + 5	0	01000	0	01000 (8)	0	8	Apagado
15 + 15	0	11110	0	11110 (30)	3	0	Apagado
9 + 9	0	10010	0	10010 (18)	1	8	Apagado
7 + 6	0	01101	0	01101 (13)	1	3	Apagado
5 − 3	1	00010	0	00010 (2)	0	2	Apagado
3 − 5	1	11110	1	00010 (2)	0	2	−
0 − 15	1	10001	1	01111 (15)	1	5	−
7 − 7	1	00000	0	00000 (0)	0	0	Apagado
15 − 0	1	01111	0	01111 (15)	1	5	Apagado

Tabla 1. Valores esperados en los buses internos del sistema para los casos de prueba de suma y resta. Calculados analíticamente con base en la lógica de complemento a 2 y decodificación BCD.

3. Códigos en Verilog HDL

Listado 1. sum1b.v, sum4b.v y sum_res_op.v – Sumador/restador de 4 bits con salida de 5 bits

```
module sum1b (
  input  A, B, Ci,
  output S, Co
);
  assign S  = A ^ B ^ Ci;
  assign Co = (A & B) | (Ci & (A ^ B));
endmodule

module sum4b (
  input  [3:0] A, B,
  input        Ci,
  output [3:0] S,
  output       Co
);
  wire c1, c2, c3;
  sum1b s0 (.A(A[0]), .B(B[0]), .Ci(Ci), .S(S[0]), .Co(c1));
  sum1b s1 (.A(A[1]), .B(B[1]), .Ci(c1), .S(S[1]), .Co(c2));
  sum1b s2 (.A(A[2]), .B(B[2]), .Ci(c2), .S(S[2]), .Co(c3));
  sum1b s3 (.A(A[3]), .B(B[3]), .Ci(c3), .S(S[3]), .Co(Co));
endmodule

module sum_res_op (
  input  [3:0] A, B,
  input        Op,      // 0 = Suma, 1 = Resta
  output [4:0] R        // Salida de 5 bits: C2 en resta, binario puro en suma
);
  wire [3:0] Bx = B ^ {4{Op}};   // Complemento a 1 de B si Op = 1
  wire [3:0] S;
  wire       Co;
  sum4b u_sum (.A(A), .B(Bx), .Ci(Op), .S(S), .Co(Co));
  assign R = {Co ^ Op, S};       // Bit R[4]: acarreo en suma, signo en resta
endmodule
```

Listado 2. detector_signo.v y calc_magnitud.v – Extracción de signo y valor absoluto

```
module detector_signo (
  input  [4:0] R,
  input        Op,
  output       signo
);
  assign signo = Op & R[4];      // Activo únicamente para restas negativas
endmodule

module calc_magnitud (
  input  [4:0] R,
  input        signo,
  output [4:0] mag
);
  assign mag = signo ? (~R + 5'd1) : R;  // Complemento a 2 si es negativo
endmodule
```

Listado 3. bin_bcd.v – Conversión binario (0 a 30) a BCD de decenas y unidades

```
module bin_bcd (
  input      [4:0] mag,
  output reg [3:0] decenas,
  output reg [3:0] unidades
);
  always @(*) begin
    if (mag >= 5'd30) begin
      decenas  = 4'd3;
      unidades = mag - 5'd30;
    end else if (mag >= 5'd20) begin
      decenas  = 4'd2;
      unidades = mag - 5'd20;
    end else if (mag >= 5'd10) begin
      decenas  = 4'd1;
      unidades = mag - 5'd10;
    end else begin
      decenas  = 4'd0;
      unidades = mag[3:0];
    end
  end
endmodule
```

Listado 4. bcd_7seg.v y display_signo.v – Decodificadores para display de ánodo común

```
module bcd_7seg (
  input      [3:0] digito,
  output reg [6:0] seg        // Orden: seg[6:0] = {a, b, c, d, e, f, g}
);
  always @(*) begin
    case (digito)
      4'd0: seg = 7'b0000001;
      4'd1: seg = 7'b1001111;
      4'd2: seg = 7'b0010010;
      4'd3: seg = 7'b0000110;
      4'd4: seg = 7'b1001100;
      4'd5: seg = 7'b0100100;
      4'd6: seg = 7'b0100000;
      4'd7: seg = 7'b0001111;
      4'd8: seg = 7'b0000000;
      4'd9: seg = 7'b0000100;
      default: seg = 7'b1111111;  // Apagado ante código inválido
    endcase
  end
endmodule

module display_signo (
  input       signo,
  output [6:0] seg_signo
);
  assign seg_signo = signo ? 7'b1111110 : 7'b1111111; // Solo segmento 'g' si es negativo
endmodule
```

Listado 5. divisor_frecuencia.v y mux_display.v – Generador de barrido y multiplexación

```
module divisor_frecuencia #(
  parameter DIV = 50000       // 50 MHz / 50000 = 1 kHz (tick de barrido)
)(
  input      clk,
  output reg tick = 1'b0
);
  reg [15:0] cnt = 16'd0;     // Registro de 16 bits (cuenta de 0 a 49999)

  always @(posedge clk) begin
    if (cnt == DIV - 1) begin
      cnt  <= 16'd0;
      tick <= 1'b1;
    end else begin
      cnt  <= cnt + 16'd1;
      tick <= 1'b0;
    end
  end
endmodule

module mux_display (
  input           clk,
  input           tick,
  input      [6:0] seg_signo,
  input      [6:0] seg_dec,
  input      [6:0] seg_uni,
  output reg [6:0] seg,
  output reg [2:0] an         // Activo en bajo: {signo, decenas, unidades}
);
  reg [1:0] sel = 2'd0;

  always @(posedge clk) begin
    if (tick)
      sel <= (sel == 2'd2) ? 2'd0 : sel + 2'd1;
  end

  always @(*) begin
    case (sel)
      2'd0: begin
        seg = seg_uni;
        an  = 3'b110;         // Habilita display de unidades
      end
      2'd1: begin
        seg = seg_dec;
        an  = 3'b101;         // Habilita display de decenas
      end
      default: begin
        seg = seg_signo;
        an  = 3'b011;         // Habilita display de signo
      end
    endcase
  end
endmodule
```

Listado 6. top.v – Integración global para tarjeta RZ-EasyFPGA A2.2

```
module top (
  input       clk,          // Pin PIN_23 (50 MHz)
  input  [3:0] A,            // Operando A
  input  [3:0] B,            // Operando B
  input       Op,           // 0 = Suma, 1 = Resta
  output [6:0] seg,          // Pines P127..P119 (segmentos a..g)
  output [2:0] an            // Pines P132, P129, P128 (ánodos de los dígitos)
);
  // NOTA: Si en el laboratorio los switches o botones son activos en bajo,
  // normalizar aqui con inversión:
  // wire [3:0] A_norm  = ~A;
  // wire [3:0] B_norm  = ~B;
  // wire       Op_norm = ~Op;
  // y conectar las señales _norm a los módulos aritméticos.

  wire [4:0] R, mag;
  wire       signo, tick;
  wire [3:0] dec, uni;
  wire [6:0] s_dec, s_uni, s_sig;

  sum_res_op u_sum (
    .A(A), .B(B), .Op(Op), .R(R)
  );

  detector_signo u_det (
    .R(R), .Op(Op), .signo(signo)
  );

  calc_magnitud u_mag (
    .R(R), .signo(signo), .mag(mag)
  );

  bin_bcd u_bcd (
    .mag(mag), .decenas(dec), .unidades(uni)
  );

  bcd_7seg u_dec (
    .digito(dec), .seg(s_dec)
  );

  bcd_7seg u_uni (
    .digito(uni), .seg(s_uni)
  );

  display_signo u_sig (
    .signo(signo), .seg_signo(s_sig)
  );

  divisor_frecuencia #(
    .DIV(50000)
  ) u_div (
    .clk(clk), .tick(tick)
  );

  mux_display u_mux (
    .clk(clk), .tick(tick),
    .seg_signo(s_sig), .seg_dec(s_dec), .seg_uni(s_uni),
    .seg(seg), .an(an)
  );
endmodule
```

Listado 7. Testbenches con autoverificación (sum_res_op, bin_bcd y top)

```
`timescale 1ns/1ps

module tb_sum_res_op;
  reg  [3:0] A, B;
  reg        Op;
  wire [4:0] R;
  integer    i, err = 0;

  sum_res_op dut (.A(A), .B(B), .Op(Op), .R(R));

  initial begin
    $dumpfile("tb_sum_res_op.vcd");
    $dumpvars(0, tb_sum_res_op);
    for (i = 0; i < 512; i = i + 1) begin
      {Op, A, B} = i[8:0];
      #10;
      if (R !== (Op ? (A - B) & 5'h1F : A + B)) begin
        err = err + 1;
        $display("ERROR: A=%d B=%d Op=%b -> R=%b", A, B, Op, R);
      end
    end
    $display("Testbench sum_res_op finalizado con %0d errores.", err);
    $finish;
  end
endmodule

module tb_bin_bcd;
  reg  [4:0] mag;
  wire [3:0] dec, uni;
  integer    k;

  bin_bcd dut (.mag(mag), .decenas(dec), .unidades(uni));

  initial begin
    $dumpfile("tb_bin_bcd.vcd");
    $dumpvars(0, tb_bin_bcd);
    for (k = 0; k <= 30; k = k + 1) begin
      mag = k[4:0];
      #10;
      if ((10 * dec + uni) != mag) begin
        $display("ERROR en conversion: mag=%d -> decenas=%d unidades=%d", mag, dec, uni);
      end
    end
    $finish;
  end
endmodule

module tb_top;
  reg        clk = 0;
  reg  [3:0] A, B;
  reg        Op;
  wire [6:0] seg;
  wire [2:0] an;

  top dut (.clk(clk), .A(A), .B(B), .Op(Op), .seg(seg), .an(an));
  defparam dut.u_div.DIV = 4; // Factor reducido para observacion rapida en simulador

  always #10 clk = ~clk;      // Periodo de 20 ns (50 MHz)

  initial begin
```

```
    $dumpfile("tb_top.vcd");
    $dumpvars(0, tb_top);
    // Caso 1: 3 + 5 = 8
    A = 3;  B = 5;  Op = 0; #400;
    // Caso 2: 15 + 15 = 30
    A = 15; B = 15; Op = 0; #400;
    // Caso 3: 3 - 5 = -2
    A = 3;  B = 5;  Op = 1; #400;
    // Caso 4: 0 - 15 = -15
    A = 0;  B = 15; Op = 1; #400;
    // Caso 5: 7 - 7 = 0
    A = 7;  B = 7;  Op = 1; #400;
    $finish;
  end
endmodule
```

4. Evidencias de simulación (GTKWave y terminal)

Las siguientes figuras contienen las capturas de pantalla de las formas de onda en GTKWave generadas a partir de los testbenches del Listado 7. Cada figura cuenta con su descripción técnica al pie.

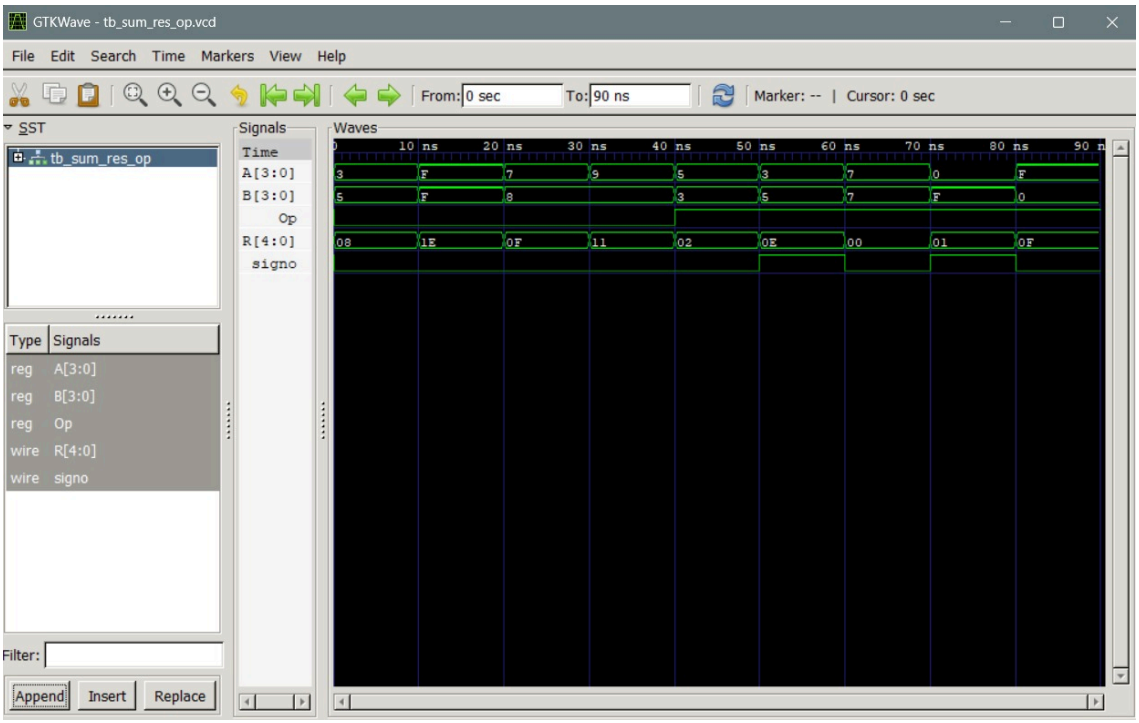

Figura 2. Simulación de sum_res_op en GTKWave. Se observa que con Op = 0 el resultado R corresponde a la adición aritmética directa (ej. 15 + 15 = 11110_2 = 30). Con Op = 1, R representa la resta en complemento a 2 (ej. 3 − 5 = 11110_2 = −2; 0 − 15 = 10001_2 = −15). La consola reporta cero errores en las 512 combinaciones evaluadas.

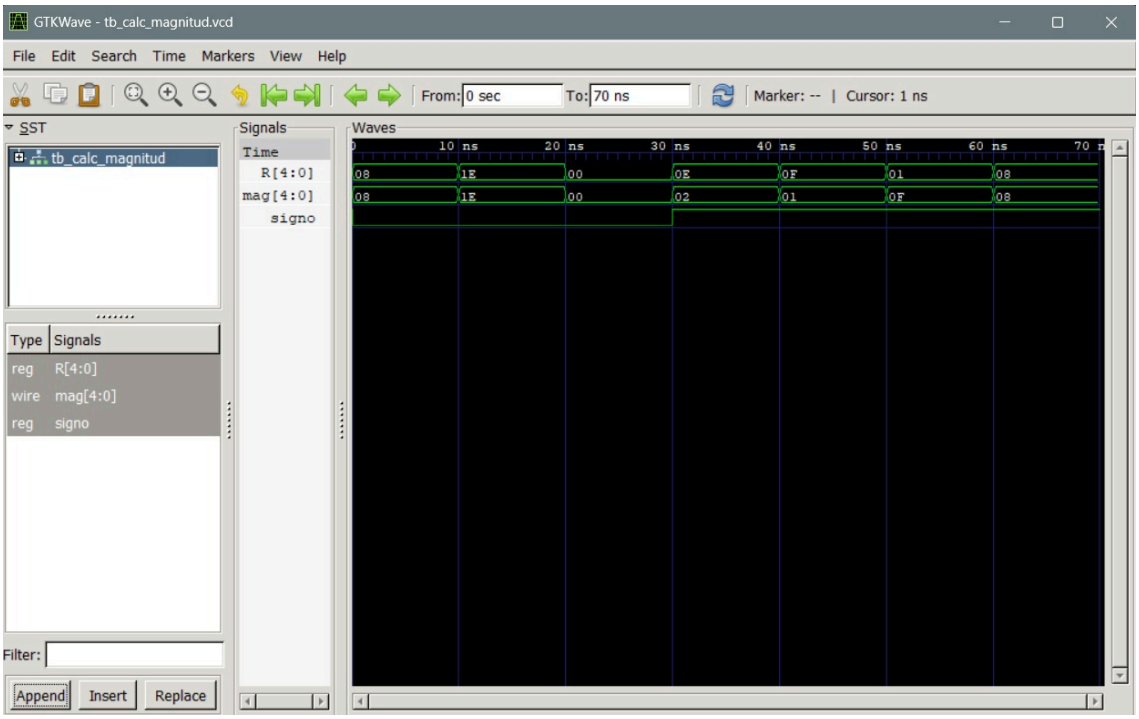

Figura 3. Simulación de detector_signo y calc_magnitud. Se verifica que cuando Op = 1 y el bit R[4] = 1, la señal signo se activa en nivel alto y mag extrae el valor absoluto mediante complemento a 2 (ej. para 0 − 15 con R = 10001, mag entrega 01111 = 15). En operaciones de adición con acarreo (15 + 15), signo se mantiene en nivel bajo a pesar de que R[4] = 1.

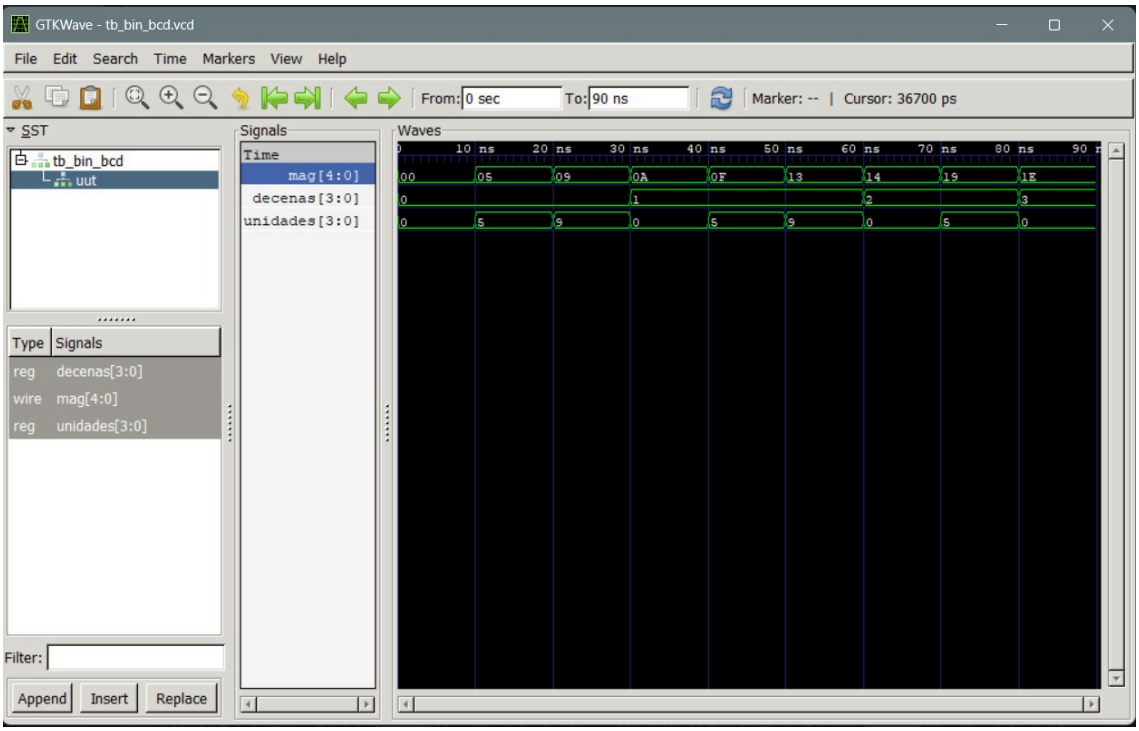

Figura 4. Simulación del conversor bin_bcd. Se ilustra la variación continua de mag desde 0 hasta 30, mostrando la transición correcta de decenas a los valores 1, 2 y 3 en los umbrales de 10, 20 y 30, mientras que unidades reinicia periódicamente en cero y avanza de forma coherente.

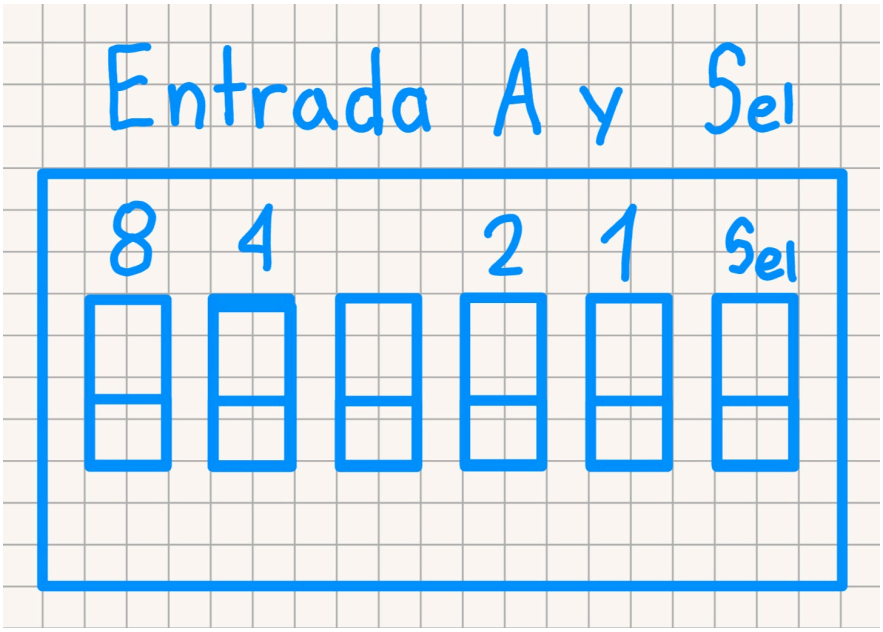

Figura 5. Representación entrada A del switch azul.

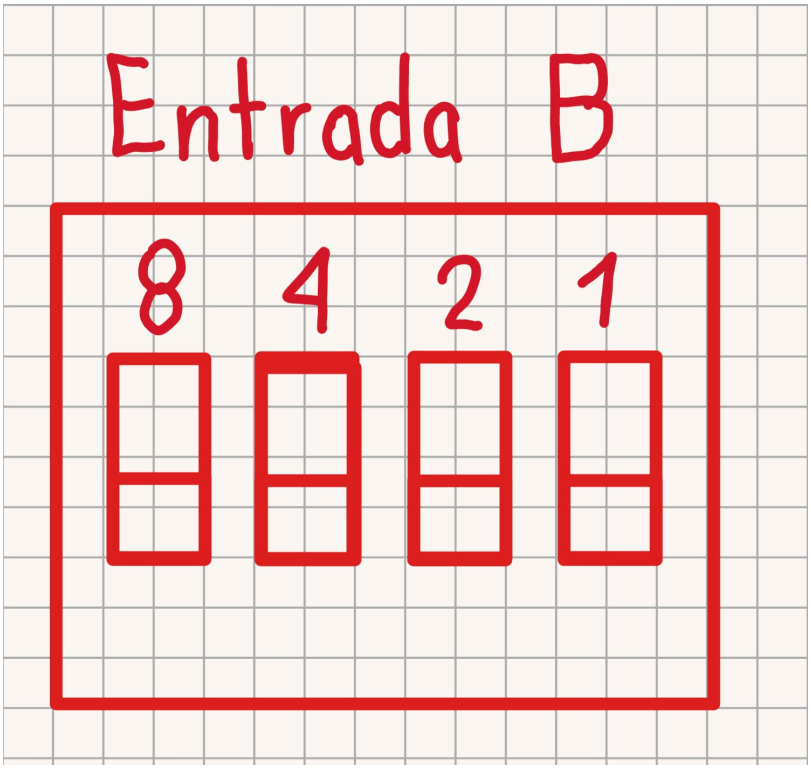

Figura 6. Representación entrada B del switch azul.

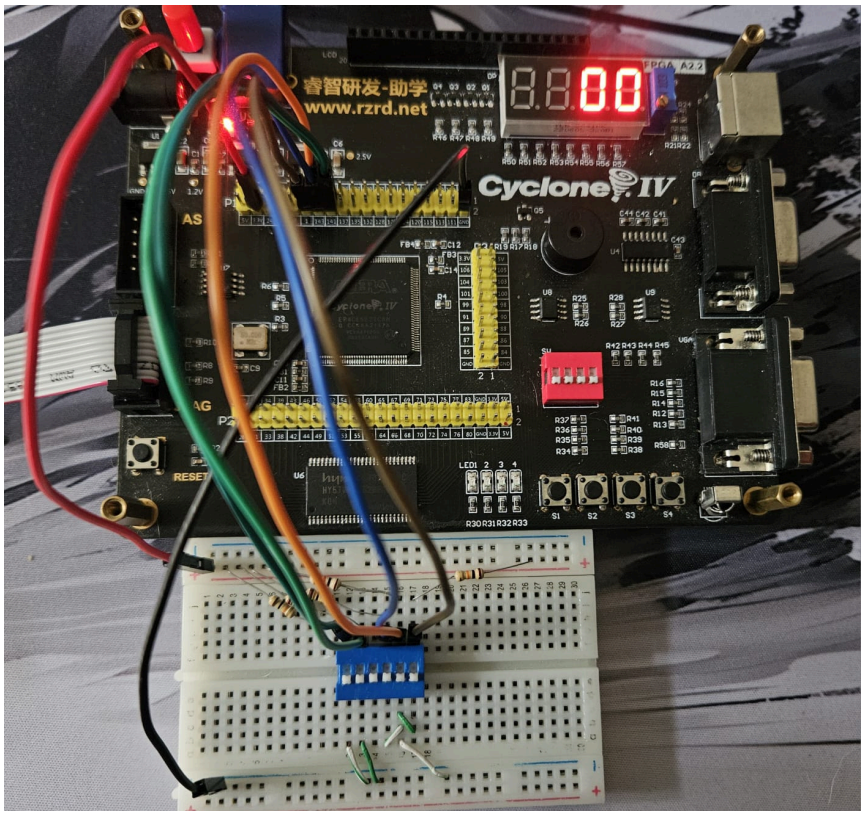

Figura 7. Resultado de la suma 0+0.

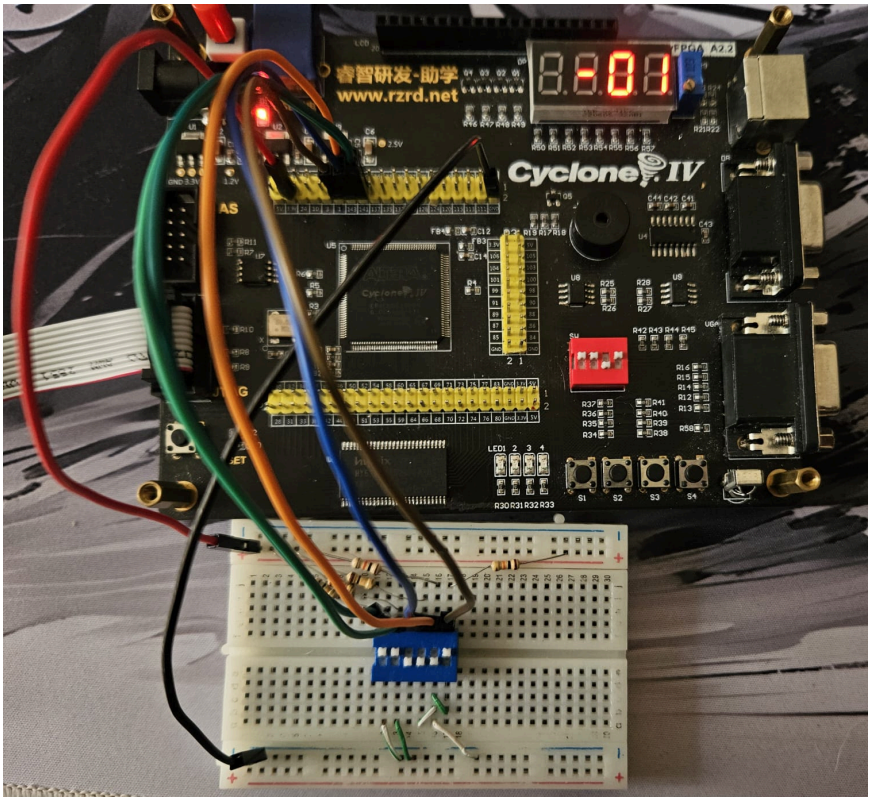

Figura 8. Resultado de la suma 12+(-13).

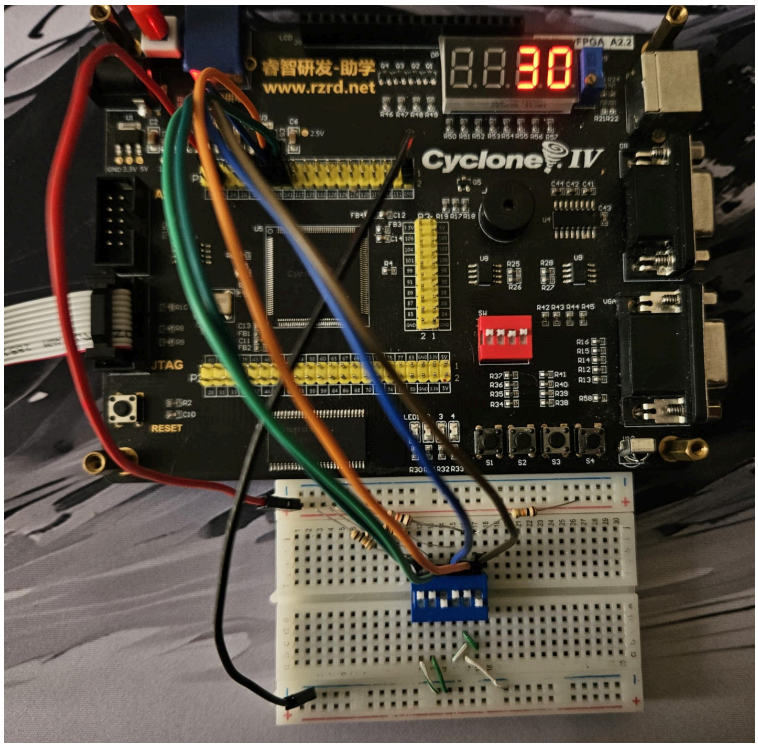

Figura 9. Resultado de la suma 15+15.

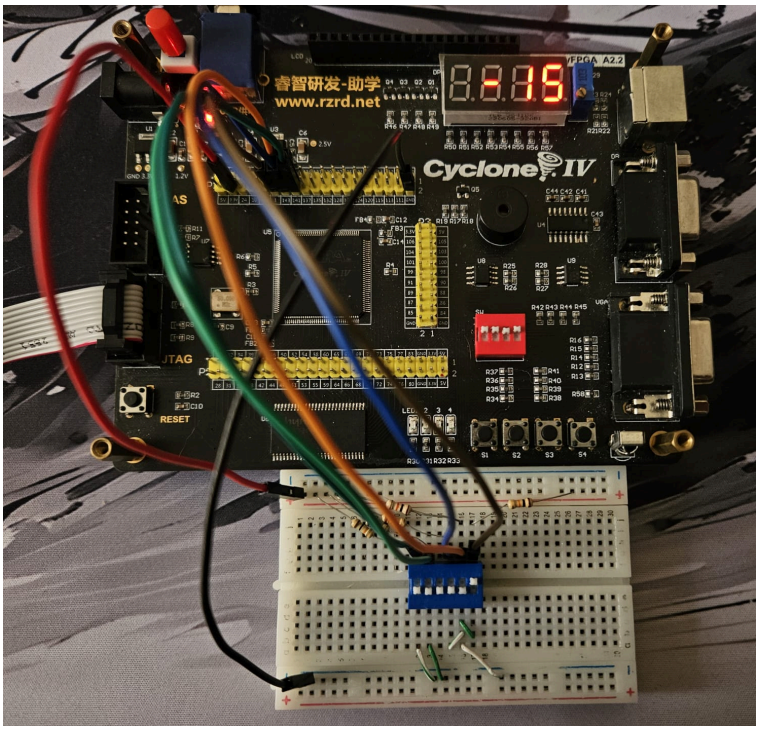

Figura 10. Resultado de la suma 0+(-15).

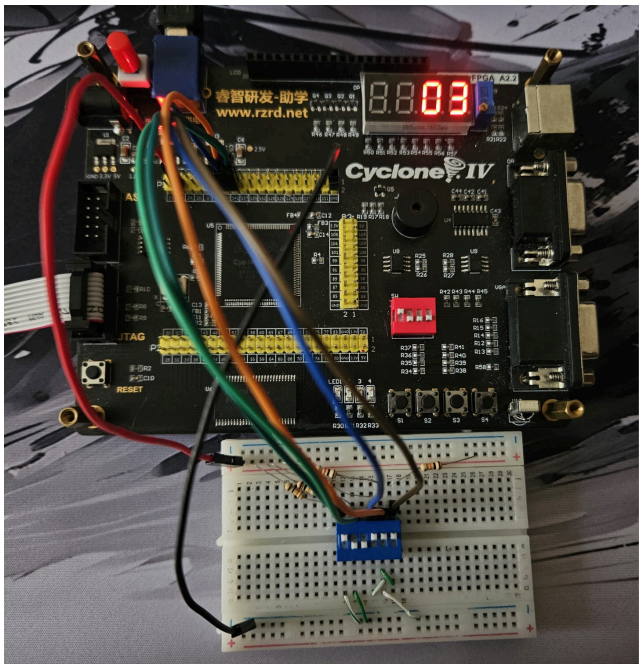

Figura 11. Resultado de la suma 11+(-8).

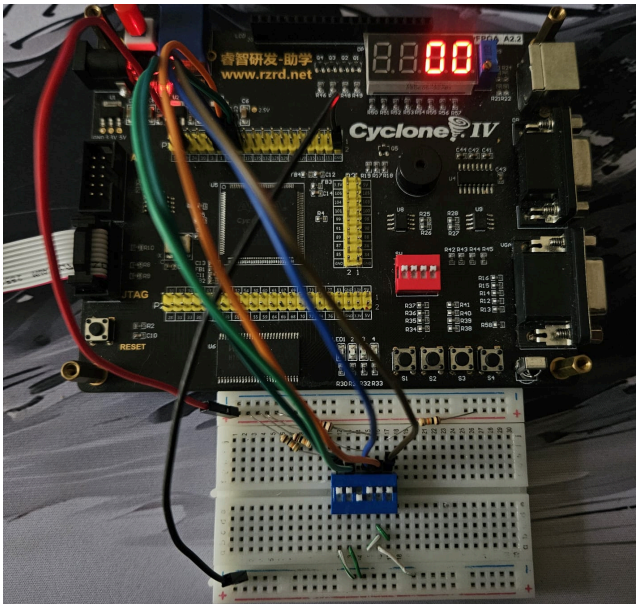

Figura 12. Resultado de la suma 15+(-15).